

Tablas de Casos de Prueba

Movies API — NestJS + TypeORM + PostgreSQL

Práctica: Pruebas Unitarias, de Integración y End-to-End

1. Pruebas Unitarias — MoviesService

Archivo: `movies.service.spec.ts` • 11 casos de prueba • Mock del repositorio con `jest.fn()`

#	Método	Comportamiento esperado	Verificación (Assert)	Tipo
Configuración				
1	—	El servicio debe estar definido después de compilar el módulo de testing	<code>service</code> no es undefined	Sanidad
create()				
2	<code>create</code>	Al recibir un DTO válido, crea la entidad y la persiste en la base de datos	<code>create()</code> y <code>save()</code> invocados con los argumentos correctos; retorna la película	Éxito
3	<code>create</code>	La película creada tiene un identificador UUID v4 asignado automáticamente	El id existe y cumple el formato regex UUID	Formato
findAll()				
4	<code>findAll</code>	Retorna todas las películas almacenadas como un array	<code>find()</code> fue invocado; resultado es un array con las películas	Éxito
5	<code>findAll</code>	Retorna un array vacío cuando no existen películas	Resultado es []	Éxito
findOne()				
6	<code>findOne</code>	Dado un UUID existente, retorna la película correspondiente	<code>findOneBy()</code> recibió { id }; resultado es la película	Éxito
7	<code>findOne</code>	Dado un UUID inexistente, lanza una excepción	Se lanza <code>NotFoundException</code>	Error
update()				
8	<code>update</code>	Dado un UUID existente y un DTO parcial, actualiza los campos y retorna la película modificada	<code>findOneBy()</code> , <code>merge()</code> y <code>save()</code> invocados correctamente; resultado refleja cambios	Éxito
9	<code>update</code>	Dado un UUID inexistente, lanza una excepción antes de intentar actualizar	Se lanza <code>NotFoundException</code>	Error
remove()				
10	<code>remove</code>	Dado un UUID existente, elimina la película de la base de datos	<code>findOneBy()</code> y <code>remove()</code> invocados con los argumentos correctos	Éxito
11	<code>remove</code>	Dado un UUID inexistente, lanza una excepción antes de intentar eliminar	Se lanza <code>NotFoundException</code>	Error

2. Pruebas de Integración — MoviesController

Archivo: `movies.controller.spec.ts` • 18 casos de prueba • Mock del servicio + `ValidationPipe` (422)

#	Método	Endpoint	Comportamiento esperado	Verificación (Assert)	Tipo
POST /movies					
1	POST	/movies	Crea una película con datos válidos	Status 201; body tiene id y title	Éxito
2	POST	/movies	Falta el campo title en el body	Status 422	Validación
3	POST	/movies	El rating es mayor a 10	Status 422	Validación
4	POST	/movies	El rating es menor a 0	Status 422	Validación
5	POST	/movies	El year es menor a 1888	Status 422	Validación
6	POST	/movies	El genre no es un valor válido del enum	Status 422	Validación
GET /movies					
7	GET	/movies	Retorna todas las películas	Status 200; body es un array con 1 elemento	Éxito
8	GET	/movies	No existen películas registradas	Status 200; body es []	Éxito
GET /movies/:id					
9	GET	/movies/:id	UUID válido y existente	Status 200; body.id coincide	Éxito
10	GET	/movies/:id	UUID con formato inválido	Status 400	Validación
11	GET	/movies/:id	UUID válido pero inexistente	Status 404	Error
PATCH /movies/:id					
12	PATCH	/movies/:id	Actualiza parcialmente con DTO válido	Status 200; body.rating refleja el cambio	Éxito
13	PATCH	/movies/:id	UUID con formato inválido	Status 400	Validación
14	PATCH	/movies/:id	UUID válido pero inexistente	Status 404	Error
15	PATCH	/movies/:id	Rating fuera de rango (ej: 15)	Status 422	Validación
DELETE /movies/:id					
16	DELETE	/movies/:id	Elimina una película existente	Status 200	Éxito
17	DELETE	/movies/:id	UUID con formato inválido	Status 400	Validación
18	DELETE	/movies/:id	UUID válido pero inexistente	Status 404	Error

3. Pruebas End-to-End — Movies E2E

Archivo: `movies.e2e-spec.ts` • 15 casos de prueba • AppModule real + base de datos de prueba + estado compartido (`createdMovieId`)

#	Método	Endpoint	Comportamiento esperado	Verificación (Assert)	Tipo
POST /movies					
1	POST	/movies	Crea una película con datos válidos y estructura completa	Status 201; body tiene id, title, director, genre, year, createdAt	Éxito
2	POST	/movies	Falta el campo title	Status 422	Validación
3	POST	/movies	El rating es 11 (fuera de rango)	Status 422	Validación
4	POST	/movies	El genre no es un valor válido	Status 422	Validación
5	POST	/movies	El year es menor a 1888	Status 422	Validación
GET /movies					
6	GET	/movies	Retorna un array que contiene la película recién creada	Status 200; body incluye un elemento con createdMovieId	Éxito
GET /movies/:id					
7	GET	/movies/:id	Obtiene la película usando createdMovieId	Status 200; body.id coincide con createdMovieId	Éxito
8	GET	/movies/:id	UUID con formato inválido	Status 400	Validación
9	GET	/movies/:id	UUID válido pero inexistente	Status 404	Error
PATCH /movies/:id					
10	PATCH	/movies/:id	Actualiza rating y synopsis con datos válidos	Status 200; body.rating es 9.0; body.synopsis es el texto actualizado	Éxito
11	PATCH	/movies/:id	UUID con formato inválido	Status 400	Validación
12	PATCH	/movies/:id	UUID válido pero inexistente	Status 404	Error
13	PATCH	/movies/:id	Rating fuera de rango (ej: 15)	Status 422	Validación
DELETE /movies/:id					
14	DELETE	/movies/:id	Elimina la película recién creada	Status 200	Éxito
15	DELETE	/movies/:id	Intenta obtener la película eliminada	Status 404 (confirma eliminación real)	Flujo

Resumen total: 44 casos de prueba

Unitarias (Service): 11 • Integración (Controller): 18 • End-to-End: 15